

DPDP COMPLIANCE AUDIT

E2EE_Cloud

Digital Personal Data Protection Act 2023



Grade B — Adequate

Scanned: 22 March 2026

Files indexed: 2

Languages: Flask, Supabase, dotenv, Python, JavaScript

CONFIDENTIAL — Internal Use Only

DPDP Compliance Scan Report

The application collects user email and password for authentication and stores user files in Supabase Storage. The biggest DPDP risk is the potential for unauthorized access to user files stored in Supabase Storage due to the lack of explicit consent mechanisms for data processing and storage.

Total findings: 5

HIGH: 1

MEDIUM: 2

INFO: 1

PASS: 1

Tech Stack: Flask, Supabase, dotenv, Python, JavaScript

Risk Profile: The application collects user email and password for authentication and stores user files in Supabase Storage. The biggest DPDP risk is the potential for unauthorized access to user files stored in Supabase Storage due to the lack of explicit consent mechanisms for data processing and storage.

Compliance score by section

DPDP Section	Weight	Score	Status
Section 6	22	0/22 (0%)	FAIL
Section 6(6)	8	8/8 (100%)	PASS
Section 8(1)	18	18/18 (100%)	PASS
Section 8	10	10/10 (100%)	PASS
Section 8(3)	8	8/8 (100%)	PASS
Section 8(4)	8	8/8 (100%)	PASS
Section 8(6)	8	4.0/8 (50%)	WARN
Section 9	6	6/6 (100%)	PASS
Section 11	6	6/6 (100%)	PASS
Section 16	4	4/4 (100%)	PASS
Section 5	2	1.0/2 (50%)	WARN
Section 13 — Grievance Redressal		4/4 (100%)	PASS

Estimated Total Remediation Effort: 0.8–2.5 engineering days

Data Flow Analysis

The following PII flow signals were inferred via static analysis (import/call adjacency). These are heuristic code-path chains, not proven runtime traces. Use them to guide developer review (confirm actual request → handler → service → database paths in your app).

Flow 1: Cloud Storage

PII fields: email, password

app.py

Rule	Section	Severity	File
CONSENT_MISSING	Section 6 — Consent	HIGH	app.py

NO_LOGGING_DETECTED	Section 8(6) — Breach	MEDIUM	N/A
THIRD_PARTY_PII_SHARING	Section 5 — Notice	MEDIUM	app.py
NO_CROSS_BORDER_DETECTE D	Section 16 — Cross-Border Transfer	INFO	N/A
DELETION_MECHANISM_PRESE NT	Section 8 — Data Principal Rights	PASS	app.py

30-Day Remediation Roadmap

The following plan prioritises findings by severity and estimated effort. Complete Week 1 items before your next security review.

Week 1 — Critical (2–8 hours)

Fix immediately — these represent the highest legal exposure.

- Consent Missing — app.py (2–8 hrs)

Week 2 — High Priority (4–12 hours)

Address within 2 weeks — short fixes with significant compliance impact.

- No Logging Detected (2–6 hrs)
- Third Party Pii Sharing — app.py (2–6 hrs)

Total estimated remediation effort: 6–20 hours (0.8–2.5 engineering days)

Details

Finding 1 of 5

[Section 6 — Consent]

1. [HIGH] CONSENT_MISSING — app.py:34**Evidence snippet:** Collects email from form data.**Estimated fix time:** 2–8 hours

Personal data collection and API endpoint detected with no consent mechanism.

Risk:

The application collects and processes user email and password without an explicit consent mechanism. This violates the requirement to obtain consent before processing personal data, potentially leading to non-compliance and user distrust.

Fix:

1. Add a checkbox or clear statement on the signup form (signup.html) requiring users to agree to terms and privacy policy before submission.
2. Modify the signup route in app.py to check for consent before calling supabase.auth.sign_up.
3. Update the privacy policy to clearly state what data is collected, why, and how it is processed by Supabase.
4. Ensure the login process also respects any consent preferences set during signup, if applicable.

DPDP:

Section 6(1) — Consent must be freely given, specific, informed, and unambiguous indication of the data subject's wishes by which he or she signifies agreement to the processing of personal data relating to him or her.

Example:

```
<!-- In signup.html --> <input type="checkbox" id="consent" name="consent" required> <label  
for="consent">I agree to the <a href="/privacy">Privacy Policy</a> and <a href="/terms">Terms  
of Service</a>.</label> # In app.py (signup route) @app.route("/signup", methods=["GET",  
"POST"]) def signup(): if request.method == "POST": email = request.form["email"] password =  
request.form["password"] consent = request.form.get("consent") # Check if consent was given if  
not consent: flash("You must agree to the terms to sign up.", "danger") return  
render_template("signup.html") # ... rest of the signup logic
```

Finding 2 of 5

[Section 8(6) — Breach]

2. [MEDIUM] NO_LOGGING_DETECTED — N / A:1**Evidence snippet:** Imports Flask, a web framework.**Estimated fix time:** 2–6 hours

No logging library detected anywhere. Breach detection and reporting may be inadequate.

Risk:

The application lacks a centralized logging mechanism, making it difficult to detect, investigate, and report on security incidents or data breaches. Without adequate logging, responding to a data breach in a timely and effective manner as required by DPDP Section 8(6) becomes significantly challenging.

Fix:

1. Install the 'Flask-Log' or 'logging' library: `pip install Flask-Log`
2. Configure a logger in app.py to capture application events, errors, and security-relevant information.
3. Integrate logging statements within routes to record key actions such as user login/logout, file access, and errors.
4. Set up log rotation and retention policies to manage log file size and ensure logs are available for a sufficient period.

DPDP:

Section 8(6) — Data controllers must implement appropriate security measures and notify the relevant supervisory authority and the data subject in the event of a personal data breach.

Example:

```
from flask import Flask, request, session import logging app = Flask(__name__) app.secret_key
= 'your secret key' # Configure logging logging.basicConfig(level=logging.INFO,
format='%(asctime)s - %(name)s - %(levelname)s - %(message)s') logger =
logging.getLogger(__name__) @app.route('/login', methods=['POST']) def login(): # ... existing
login code ... try: # ... successful login ... logger.info(f"User logged in: {email}")
session["user"] = {"id": user.user.id, "email": user.user.email} return
redirect(url_for("storage")) except Exception as e: logger.error(f"Login failed for {email}:
{e}") flash("Invalid credentials", "danger") return render_template("login.html")
```

Finding 3 of 5

[Section 5 — Notice]

3. [MEDIUM] THIRD_PARTY_PII_SHARING — app.py:2**Evidence snippet:** Imports the Supabase client library, which may handle or transmit PII.**Estimated fix time:** 2–6 hours

Personal data present in file that imports third-party SDK. Data may be shared without user awareness or notice (DPDP Section 5).

Risk:

The application uses the Supabase SDK to process user credentials (email and password), which constitutes sharing personal data with a third party. This sharing may occur without adequate notice to the user, violating DPDP Section 5 requirements for transparency.

Fix:

1. Update the privacy policy to explicitly mention the use of Supabase for authentication and data storage, detailing the types of data shared and the purpose.
2. Add a clear statement on the signup and login pages informing users that their data will be processed by Supabase according to the privacy policy.
3. Review Supabase's data processing agreements and security measures to ensure they align with DPDP requirements.
4. Consider adding a link to the privacy policy near the consent checkbox on the signup form.

DPDP:

Section 5(1) — A data controller must provide clear and accessible notice to data subjects about the collection, use, and disclosure of their personal data.

Example:

```
<!-- In signup.html --> <p>By signing up, you agree to our <a href="/privacy">Privacy
Policy</a>, which explains how we use Supabase to process your data.</p> <input
type="checkbox" id="consent" name="consent" required> <label for="consent">I agree</label> #
In app.py (signup route) @app.route("/signup", methods=["GET", "POST"]) def signup(): if
request.method == "POST": email = request.form["email"] password = request.form["password"]
consent = request.form.get("consent") if not consent: flash("You must agree to the privacy
policy.", "danger") return render_template("signup.html") # ... rest of signup logic using
supabase ...
```

Finding 4 of 5

[Section 16 — Cross-Border Transfer]

4. [INFO] NO_CROSS_BORDER_DETECTED — N / A

No foreign cloud or non-India region config detected.

Finding 5 of 5

[Section 8 — Data Principal Rights]

5. [PASS] DELETION_MECHANISM_PRESENT — app.py

Data deletion mechanism detected. Found 1 deletion endpoint(s) and 1 deletion signal(s). Note: Verify deletion cascades to all PII tables (sessions, logs, related records).

Fix:

1. Verify deletion cascades to: sessions, audit_logs, third-party sync records.
2. Use database CASCADE DELETE or explicit multi-table deletion in deletion service.
3. Test deletion completeness: create account, delete, query all PII tables for orphaned data.

AI-Identified Compliance Gaps

The following gaps were identified by AI analysis and cross-validated. These are advisory observations, not verified rule violations. Human review is required before acting on these findings.

[AI-INFERRED] Missing automated data export functionality

Confidence: 60%

Section 8 — Right to Access

The application stores user email and ID in session (``app.py:61-62``) and user files in Supabase Storage. While a ``GET /download`` endpoint exists for individual files, there is no explicit endpoint or mechanism to export **all** personal data associated with a user account (e.g., account details, metadata, and an archive of all uploaded files) in a machine-readable format, as required by Section 8(1)(c). The existing ``dpdp_coverage_gaps`` mentions 'No mechanism for data access or correction requests visible', which refers to the request process, not the technical implementation of an automated data export.

Recommendation: Implement a new endpoint (e.g., ``/data_export``) that allows authenticated users to download an archive containing all their personal data, including account details (email, ID) and an archive of all associated files/metadata, in a structured, commonly used, machine-readable format.

[AI-INFERRED] No explicit consent withdrawal or account deletion

Confidence: 65%

Section 6(6) — Consent Withdrawal

The application handles user authentication and file storage. While the rule engine flagged ``CONSENT_MISSING`` for initial consent collection, there is no explicit endpoint or functionality (e.g., ``/delete_account``) that allows a user to withdraw their consent for data processing and storage, or to fully delete their account and all associated personal data (email, ID, and all uploaded files). The ``GET /delete`` endpoint only handles individual file deletion, not comprehensive account and data removal.

Recommendation: Add a ``/delete_account`` endpoint that, upon user confirmation, removes the user's account from Supabase Auth, deletes all associated PII (email, ID from session/database), and removes all user-uploaded files from Supabase Storage, thereby facilitating consent withdrawal.

[AI-INFERRED] No age verification for account creation

Confidence: 60%

Section 9 — Children

The ``POST /signup`` endpoint in ``app.py`` collects user email and password for account creation. The repository context does not indicate any mechanism to collect or verify the age of the user during the signup process, which is a requirement under Section 9 of the DPDP Act for processing children's data.

Recommendation: Modify the ``/signup`` endpoint to include an age collection field and implement age verification logic to prevent minors from creating accounts without verifiable parental consent, or to restrict services for children as appropriate.

[AI-INFERRED] Missing Data Protection Officer contact information

Confidence: 35%

Section 10 — Data Protection Officer

The repository context does not indicate any public-facing contact information for a Data Protection Officer (DPO) or a designated grievance officer. There is no mention of a privacy policy page, contact page, or environment variable that would typically contain this information, which is a requirement under Section 10 of the DPDP Act for Data Fiduciaries.

Recommendation: Add a dedicated section (e.g., in a privacy policy page, contact page, or application footer) that clearly states the contact details for the Data Protection Officer or a designated grievance officer.

Deep Compliance Review

Overall Risk: CRITICAL

The application exhibits critical architectural flaws that severely compromise user data protection, primarily due to running in an insecure debug mode in production and failing to enforce server-side encryption for user-uploaded files. This creates a high risk of unauthorized access and exposure of personal data stored on third-party services. Furthermore, the direct embedding of PII in storage metadata and the absence of robust data retention policies exacerbate these risks, indicating a systemic lack of data protection by design and by default.

Most Critical Action: Disable debug mode in production environments across all application layers to prevent information disclosure and unauthorized access.

- **Debug Mode Enabled in Production** [HIGH] — Security of processing (Article 32) — layers: Third-Party & PII Flows, API Routes & Controllers, Configuration & Secrets
- **PII in Storage Object Paths/Names** [MEDIUM] — Data minimization (Article 5(1)(c)), Pseudonymization (Article 25) — layers: Third-Party & PII Flows, API Routes & Controllers
- **Inadequate Data Protection for User Files at Rest** [HIGH] — Security of processing (Article 32), Data protection by design and by default (Article 25) — layers: Third-Party & PII Flows

1. [HIGH] Debug Mode Enabled in Production — N/A (application configuration) — Security of processing (Article 32)

The Flask application is configured to run with `debug=True` when executed directly (`if \_\_name\_\_ == "\_\_main\_\_": app.run(debug=True)`), appearing in both Third-Party & PII Flows and API Routes & Controllers contexts.

2. [HIGH] Unencrypted User Files Stored on Third-Party — N/A (upload route logic) — Security of processing (Article 32), Data protection by design and by default (Article 25)

The `/upload` route directly stores user-uploaded files to Supabase storage without server-side encryption enforcement.

3. [HIGH] Flawed Client-Side Encryption — encryption.js — Security of processing (Article 32), Data protection by design and by default (Article 25)

The `encryption.js` script attempts client-side encryption, but the generated `salt` is not stored or transmitted with the encrypted data, making decryption impossible.

4. [MEDIUM] PII in Storage Object Paths/Names — N/A (storage path generation logic) — Data minimization (Article 5(1)(c)), Pseudonymization (Article 25)

User email (after sanitization), original filenames, and folder names are directly incorporated into Supabase storage object paths and names.

5. [MEDIUM] Missing Data Retention Policy Enforcement — N/A (application logic) — Storage limitation (Article 5(1)(e)), Data protection by design and by default (Article 25)
There is no explicit data retention policy enforcement for user-uploaded content.

Layer Breakdown

Third-Party & PII Flows — 4 finding(s). This layer exhibits critical DPDP compliance failures, primarily concerning the security of personal data at rest and in transit. The application runs in an insecure debug mode, fails to enforce encryption for user-uploaded content, and contains a fundamentally broken client-side encryption mechanism, creating a significant risk of PII exposure. Additionally, PII is embedded in storage metadata, complicating data minimization and management.

API Routes & Controllers — 4 finding(s). This API Routes & Controllers layer exhibits critical security vulnerabilities, notably running in debug mode in a production context, which severely compromises data confidentiality. Additionally, the layer lacks robust data lifecycle management, with no explicit data retention policies for user-uploaded content. Generic error handling also poses an information disclosure risk, and the direct use of PII in storage paths raises concerns regarding data minimization and pseudonymization.